

Environment and how to run

A single file, `lecture12_exercise.py` — runs on **CPU**, no GPU needed

Download `lecture12_exercise.py` and `requirements.txt` from the course repository

Implement each function marked `TODO`

Run the file to self-check against the built-in tests

```
pip install -r requirements.txt
python lecture12_exercise.py
```

github.com/mpkato/ir_lecture_slides → `lecture12_exercise.py` — the first run downloads the models (~420 MB) from Hugging Face

Real models on a small corpus

An 11-document corpus with a built-in **vocabulary mismatch** (e.g., *car fuel efficiency* vs. "automobile ... gasoline ... mileage"). You write the code that runs three **real pre-trained models**:

Model	Tasks
cross-encoder/ ms-marco-MiniLM-L6-v2	1-2
doc2query/ msmarco-t5-small-v1	3
sentence-transformers/ all-MiniLM-L6-v2	4-5

Task	Lecture	Points
Task 1 — Cross-encoder scoring	Lectures 7–8	20
Task 2 — Reranking (Retrieve-and-Rerank, Stage 2)	Lecture 7	20
Task 3 — doc2query query generation	Lecture 9	20
Task 4 — Bi-encoder embedding	Lecture 10	20
Task 5 — Dense retrieval	Lecture 10	20
Total		100

All tasks are checked by the built-in tests — they also assert the lecture take-aways, e.g., the cross-encoder demotes an off-topic decoy that BM25 ranks high, and dense retrieval finds a document that shares no term with the query

Deliverables and naming

One file, named **starting with your student number:**

1. Your completed `lecture12_exercise.py`

```
20XXXXXXX_lecture12_exercise.py
```

Where and when

Upload the file to the course **Dropbox file request:**

Before the second lecture from today begins

Note on generative AI — Tasks are checked by **hidden tests** beyond those shown, with different queries and documents; hard-coding the visible outputs will not pass. Your functions must actually call the specified models